

Backend Schema (Supabase)

Food Booking System

Food Booking System · Jesam Obona

Backend Schema (Supabase / PostgreSQL)

Food Booking System

Note: This schema assumes the slot-based pickup model and prepay-on-order flow described in ADR-001 and ADR-002 (Implementation Plan). If field research points the other way, `pickup_slots` and `payments / orders.payment_status` are the sections to revisit — the rest of the schema is largely unaffected either way.

1. Entity Overview

```
auth.users (Supabase-managed)
├─ admin_users
├─ vendor_accounts → vendors → menu_items
│   └─ pickup_slots
└─ customers

orders → order_items → menu_items
orders → payments
orders → vendors, pickup_slots, customers
```

2. Tables

```

-- =====
-- Food Booking System – Core Schema
-- =====

-- 1. VENDORS (the business entity)
CREATE TABLE IF NOT EXISTS vendors (
  id BIGSERIAL PRIMARY KEY,
  name TEXT NOT NULL,
  description TEXT,
  logo_url TEXT,
  area TEXT,                -- e.g. "Wuse 2", "Area 1" – used for discovery filtering
  address TEXT,
  phone TEXT,
  avg_prep_time_minutes INTEGER DEFAULT 15,
  status TEXT NOT NULL DEFAULT 'pending'
  CHECK (status IN ('pending', 'active', 'suspended', 'rejected')),
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- 2. ADMIN USERS
CREATE TABLE IF NOT EXISTS admin_users (
  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  email TEXT NOT NULL UNIQUE,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- 3. VENDOR ACCOUNTS (owner/staff who manage a vendor)
CREATE TABLE IF NOT EXISTS vendor_accounts (
  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  vendor_id BIGINT NOT NULL REFERENCES vendors(id) ON DELETE CASCADE,
  display_name TEXT,
  role TEXT NOT NULL DEFAULT 'owner' CHECK (role IN ('owner', 'staff')),
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_vendor_accounts_vendor_id ON vendor_accounts(vendor_id);

-- 4. CUSTOMERS (lightweight profile on top of auth.users)
CREATE TABLE IF NOT EXISTS customers (
  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  name TEXT,
  phone TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- 5. MENU ITEMS (today's available items – not a fixed permanent catalog)
CREATE TABLE IF NOT EXISTS menu_items (
  id BIGSERIAL PRIMARY KEY,
  vendor_id BIGINT NOT NULL REFERENCES vendors(id) ON DELETE CASCADE,
  name TEXT NOT NULL,
  description TEXT,
  photo_url TEXT,
  price NUMERIC(10, 2) NOT NULL CHECK (price >= 0),
  quantity_remaining INTEGER,        -- NULL = unlimited/not stock-tracked, else decremented per order
  is_available BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_menu_items_vendor_id ON menu_items(vendor_id);
CREATE INDEX IF NOT EXISTS idx_menu_items_available ON menu_items(vendor_id, is_available);

-- 6. PICKUP SLOTS (per ADR-001 – fixed windows with capacity)
CREATE TABLE IF NOT EXISTS pickup_slots (
  id BIGSERIAL PRIMARY KEY,
  vendor_id BIGINT NOT NULL REFERENCES vendors(id) ON DELETE CASCADE,
  slot_date DATE NOT NULL,
  start_time TIME NOT NULL,
  end_time TIME NOT NULL,
  capacity INTEGER NOT NULL DEFAULT 20 CHECK (capacity > 0),
  orders_count INTEGER NOT NULL DEFAULT 0 CHECK (orders_count >= 0),
  created_at TIMESTAMPTZ DEFAULT NOW(),

```

```

    CHECK (end_time > start_time)
);

CREATE INDEX IF NOT EXISTS idx_pickup_slots_vendor_date ON pickup_slots(vendor_id, slot_date);

-- 7. ORDERS
CREATE TABLE IF NOT EXISTS orders (
    id BIGSERIAL PRIMARY KEY,
    customer_id UUID NOT NULL REFERENCES customers(id),
    vendor_id BIGINT NOT NULL REFERENCES vendors(id),
    pickup_slot_id BIGINT REFERENCES pickup_slots(id),
    status TEXT NOT NULL DEFAULT 'pending'
    CHECK (status IN ('pending', 'accepted', 'rejected', 'preparing', 'ready', 'collected', 'cancelled', 'no_show')),
    pickup_code TEXT NOT NULL, -- short human-facing code, e.g. 6-digit
    total_amount NUMERIC(10, 2) NOT NULL CHECK (total_amount >= 0),
    payment_status TEXT NOT NULL DEFAULT 'unpaid'
    CHECK (payment_status IN ('unpaid', 'paid', 'refunded', 'failed')),
    idempotency_key TEXT NOT NULL UNIQUE, -- prevents duplicate order creation on retry
    rejection_reason TEXT,
    accepted_at TIMESTAMPTZ,
    ready_at TIMESTAMPTZ,
    collected_at TIMESTAMPTZ,
    cancelled_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_orders_customer_id ON orders(customer_id);
CREATE INDEX IF NOT EXISTS idx_orders_vendor_id ON orders(vendor_id, status);
CREATE UNIQUE INDEX IF NOT EXISTS idx_orders_pickup_code_active
    ON orders(vendor_id, pickup_code)
    WHERE status NOT IN ('collected', 'cancelled', 'rejected', 'no_show');

-- 8. ORDER ITEMS (price/name snapshot at order time – never reference live menu_items price)
CREATE TABLE IF NOT EXISTS order_items (
    id BIGSERIAL PRIMARY KEY,
    order_id BIGINT NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
    menu_item_id BIGINT REFERENCES menu_items(id), -- kept for traceability; nullable in case item is later deleted
    name_snapshot TEXT NOT NULL,
    price_snapshot NUMERIC(10, 2) NOT NULL CHECK (price_snapshot >= 0),
    quantity INTEGER NOT NULL CHECK (quantity > 0),
    subtotal NUMERIC(10, 2) NOT NULL CHECK (subtotal >= 0)
);

CREATE INDEX IF NOT EXISTS idx_order_items_order_id ON order_items(order_id);

-- 9. PAYMENTS (one row per payment attempt/confirmation)
CREATE TABLE IF NOT EXISTS payments (
    id BIGSERIAL PRIMARY KEY,
    order_id BIGINT NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
    provider TEXT NOT NULL CHECK (provider IN ('paystack', 'flutterwave')),
    provider_reference TEXT NOT NULL,
    amount NUMERIC(10, 2) NOT NULL,
    status TEXT NOT NULL DEFAULT 'pending' CHECK (status IN ('pending', 'success', 'failed')),
    raw_payload JSONB, -- full webhook payload for audit/debugging
    created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE UNIQUE INDEX IF NOT EXISTS idx_payments_provider_reference ON payments(provider, provider_reference);
CREATE INDEX IF NOT EXISTS idx_payments_order_id ON payments(order_id);

-- 10. NOTIFICATIONS LOG (audit trail for push/SMS sends)
CREATE TABLE IF NOT EXISTS notifications_log (
    id BIGSERIAL PRIMARY KEY,
    user_id UUID NOT NULL,
    order_id BIGINT REFERENCES orders(id),
    channel TEXT NOT NULL CHECK (channel IN ('push', 'sms')),
    message TEXT NOT NULL,
    status TEXT NOT NULL DEFAULT 'sent' CHECK (status IN ('sent', 'failed')),
    created_at TIMESTAMPTZ DEFAULT NOW()
);

```

3. Atomic Order Placement (RPC function)

Prevents overselling and enforces idempotency in a single transaction — call this via Supabase RPC from the API route rather than doing separate insert/update calls from the client or route handler.

```

CREATE OR REPLACE FUNCTION place_order(
  p_customer_id UUID,
  p_vendor_id BIGINT,
  p_pickup_slot_id BIGINT,
  p_idempotency_key TEXT,
  p_items JSONB -- [{menu_item_id, quantity}, ...]
) RETURNS BIGINT
LANGUAGE plpgsql
AS $$
DECLARE
  v_order_id BIGINT;
  v_item JSONB;
  v_menu_item RECORD;
  v_total NUMERIC(10,2) := 0;
  v_subtotal NUMERIC(10,2);
  v_pickup_code TEXT;
BEGIN
  -- Idempotency check
  SELECT id INTO v_order_id FROM orders WHERE idempotency_key = p_idempotency_key;
  IF FOUND THEN
    RETURN v_order_id;
  END IF;

  -- Slot capacity check (if using slot model)
  IF p_pickup_slot_id IS NOT NULL THEN
    UPDATE pickup_slots
      SET orders_count = orders_count + 1
      WHERE id = p_pickup_slot_id AND orders_count < capacity;
    IF NOT FOUND THEN
      RAISE EXCEPTION 'PICKUP_SLOT_FULL';
    END IF;
  END IF;

  v_pickup_code := LPAD(FLOOR(RANDOM() * 999999)::TEXT, 6, '0');

  INSERT INTO orders (customer_id, vendor_id, pickup_slot_id, pickup_code, total_amount, idempotency_key)
  VALUES (p_customer_id, p_vendor_id, p_pickup_slot_id, v_pickup_code, 0, p_idempotency_key)
  RETURNING id INTO v_order_id;

  FOR v_item IN SELECT * FROM jsonb_array_elements(p_items)
  LOOP
    SELECT * INTO v_menu_item FROM menu_items
      WHERE id = (v_item->>'menu_item_id')::BIGINT
      FOR UPDATE; -- row lock to prevent concurrent oversell

    IF v_menu_item.is_available = FALSE THEN
      RAISE EXCEPTION 'ITEM_UNAVAILABLE: %', v_menu_item.name;
    END IF;

    IF v_menu_item.quantity_remaining IS NOT NULL THEN
      IF v_menu_item.quantity_remaining < (v_item->>'quantity')::INTEGER THEN
        RAISE EXCEPTION 'INSUFFICIENT_STOCK: %', v_menu_item.name;
      END IF;
      UPDATE menu_items SET quantity_remaining = quantity_remaining - (v_item->>'quantity')::INTEGER
        WHERE id = v_menu_item.id;
    END IF;

    v_subtotal := v_menu_item.price * (v_item->>'quantity')::INTEGER;
    v_total := v_total + v_subtotal;

    INSERT INTO order_items (order_id, menu_item_id, name_snapshot, price_snapshot, quantity, subtotal)
    VALUES (v_order_id, v_menu_item.id, v_menu_item.name, v_menu_item.price, (v_item->>'quantity')::INTEGER, v_subtotal);
  END LOOP;

  UPDATE orders SET total_amount = v_total WHERE id = v_order_id;

  RETURN v_order_id;
END;
$$;

```

4. `updated_at` Trigger (reusable)

```
CREATE OR REPLACE FUNCTION set_updated_at()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_vendors_updated_at BEFORE UPDATE ON vendors
FOR EACH ROW EXECUTE FUNCTION set_updated_at();
CREATE TRIGGER trg_menu_items_updated_at BEFORE UPDATE ON menu_items
FOR EACH ROW EXECUTE FUNCTION set_updated_at();
CREATE TRIGGER trg_orders_updated_at BEFORE UPDATE ON orders
FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

5. Row Level Security

```

ALTER TABLE vendors ENABLE ROW LEVEL SECURITY;
ALTER TABLE vendor_accounts ENABLE ROW LEVEL SECURITY;
ALTER TABLE customers ENABLE ROW LEVEL SECURITY;
ALTER TABLE menu_items ENABLE ROW LEVEL SECURITY;
ALTER TABLE pickup_slots ENABLE ROW LEVEL SECURITY;
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;
ALTER TABLE order_items ENABLE ROW LEVEL SECURITY;
ALTER TABLE payments ENABLE ROW LEVEL SECURITY;

-- Vendors: public can view active vendors (discovery); vendor/admin can manage
CREATE POLICY vendors_read_public ON vendors
  FOR SELECT USING (status = 'active' OR EXISTS (
    SELECT 1 FROM vendor_accounts WHERE id = auth.uid() AND vendor_id = vendors.id
  ) OR EXISTS (SELECT 1 FROM admin_users WHERE id = auth.uid()));

CREATE POLICY vendors_write_admin_or_owner ON vendors
  FOR UPDATE USING (
    EXISTS (SELECT 1 FROM admin_users WHERE id = auth.uid())
    OR EXISTS (SELECT 1 FROM vendor_accounts WHERE id = auth.uid() AND vendor_id = vendors.id AND role = 'owner')
  );

-- Menu items: public can view available items of active vendors; vendor manages own
CREATE POLICY menu_items_read_public ON menu_items
  FOR SELECT USING (
    EXISTS (SELECT 1 FROM vendors WHERE id = menu_items.vendor_id AND status = 'active')
    OR EXISTS (SELECT 1 FROM vendor_accounts WHERE id = auth.uid() AND vendor_id = menu_items.vendor_id)
    OR EXISTS (SELECT 1 FROM admin_users WHERE id = auth.uid())
  );

CREATE POLICY menu_items_write_vendor ON menu_items
  FOR ALL USING (
    EXISTS (SELECT 1 FROM vendor_accounts WHERE id = auth.uid() AND vendor_id = menu_items.vendor_id)
    OR EXISTS (SELECT 1 FROM admin_users WHERE id = auth.uid())
  );

-- Pickup slots: public read, vendor writes own
CREATE POLICY pickup_slots_read_public ON pickup_slots FOR SELECT USING (true);
CREATE POLICY pickup_slots_write_vendor ON pickup_slots
  FOR ALL USING (
    EXISTS (SELECT 1 FROM vendor_accounts WHERE id = auth.uid() AND vendor_id = pickup_slots.vendor_id)
    OR EXISTS (SELECT 1 FROM admin_users WHERE id = auth.uid())
  );

-- Orders: customer sees own, vendor sees own vendor's, admin sees all
CREATE POLICY orders_read_own ON orders
  FOR SELECT USING (
    customer_id = auth.uid()
    OR EXISTS (SELECT 1 FROM vendor_accounts WHERE id = auth.uid() AND vendor_id = orders.vendor_id)
    OR EXISTS (SELECT 1 FROM admin_users WHERE id = auth.uid())
  );

CREATE POLICY orders_insert_own ON orders
  FOR INSERT WITH CHECK (customer_id = auth.uid());

CREATE POLICY orders_update_vendor_or_admin ON orders
  FOR UPDATE USING (
    EXISTS (SELECT 1 FROM vendor_accounts WHERE id = auth.uid() AND vendor_id = orders.vendor_id)
    OR EXISTS (SELECT 1 FROM admin_users WHERE id = auth.uid())
    OR customer_id = auth.uid() -- customer can update own order only for cancellation (enforce allowed-status transition in
app logic)
  );

-- Order items: readable if parent order is readable
CREATE POLICY order_items_read ON order_items
  FOR SELECT USING (
    EXISTS (
      SELECT 1 FROM orders
      WHERE orders.id = order_items.order_id
      AND (
        orders.customer_id = auth.uid()
        OR EXISTS (SELECT 1 FROM vendor_accounts WHERE id = auth.uid() AND vendor_id = orders.vendor_id)
        OR EXISTS (SELECT 1 FROM admin_users WHERE id = auth.uid())
      )
    )
  );

```

```

    )
  );

-- Payments: readable by order owner/vendor/admin; writes reserved for service role (webhook handler)
CREATE POLICY payments_read ON payments
  FOR SELECT USING (
    EXISTS (
      SELECT 1 FROM orders
      WHERE orders.id = payments.order_id
      AND (
        orders.customer_id = auth.uid()
        OR EXISTS (SELECT 1 FROM vendor_accounts WHERE id = auth.uid() AND vendor_id = orders.vendor_id)
        OR EXISTS (SELECT 1 FROM admin_users WHERE id = auth.uid())
      )
    )
  );

-- No general INSERT/UPDATE policy on payments – only the service-role webhook handler writes here,
-- bypassing RLS via the service role key (mirrors the pattern used for /api/sync/* routes).

```

6. Design Notes

- **quantity_remaining is nullable by design** — some vendors will track exact stock ("only 12 plates of rice left"), others will just toggle items on/off. Both models are supported without a schema change.
- **idempotency_key is unique** — a retried checkout request (e.g. after a network drop) resolves to the same order rather than creating a duplicate.
- **Price is snapshotted onto order_items** — a vendor changing today's price never rewrites yesterday's order history.
- **payments writes go through the service role only** — client-reported payment success is never trusted; only a verified webhook confirms payment (see TRD §4, Reliability).
- **Pickup code uniqueness** is scoped per vendor and only among active orders, so codes can be short (6 digits) without risking collisions with historical orders.